

大实验报告——AgeNet (APPA-REAL) 年龄预测

姓名：李祥熙 学号：2234412799

姓名：储正非 学号：2233412534

姓名：袁宇霄 学号：2235015935

2026 年 6 月 5 日

目录

1 实验目的	3
2 实验过程	3
2.1 实验环境	3
2.2 年龄估计建模方法	3
2.3 评价指标	4
2.4 数据集与预处理	4
2.4.1 数据集	4
2.4.2 基础预处理	5
2.4.3 训练集增强	5
2.5 实现步骤	5
2.5.1 实现 AgeNet	5
2.5.2 数据集加载、训练和验证	6
3 实验内容	7
3.1 模型结构与关键实现	7
3.2 训练流程（关键片段）	8
4 实验结果	8
4.1 训练日志摘录	8
4.2 结果分析	9
5 总结	10

6 附录：主要源码与运行命令	10
6.1 运行命令	10
6.2 主要源码	10
6.2.1 age_net.py	10
6.2.2 train_ageNet.py	12

1 实验目的

本实验完成一个基于 APPA-REAL 人脸数据集的端到端年龄预测系统。核心任务是在 PyTorch 中实现指定的 `AgeNet(nn.Module)`，输入 128×128 RGB 人脸图像，输出 0 到 100 岁共 101 个年龄类别。

本次实验的主要目标如下：

1. 理解年龄估计任务中分类建模与回归评价相结合的思想；
2. 使用 PyTorch 实现课程要求的 AgeNet 卷积神经网络；
3. 完成 APPA-REAL 图像数据读取、预处理、数据增强、训练、验证和测试预测；
4. 使用平均绝对误差 MAE 评价年龄估计结果；
5. 保存验证集 MAE 最优的模型权重，并输出测试集预测年龄文件；
6. 根据训练日志分析模型收敛过程和实验结果。

2 实验过程

2.1 实验环境

- 操作系统：Linux
- 编程语言：Python 3
- 深度学习框架：PyTorch
- 关键依赖：torchvision, pandas, Pillow, OpenCV, imgaug, numpy
- 计算资源：优先使用 GPU（训练日志显示使用了 CUDA）

2.2 年龄估计建模方法

年龄估计既可以直接建模为回归问题，也可以转化为分类问题。本实验采用分类训练、期望回归的方式：模型输出 101 维 logits，每一维对应一个年龄类别。训练阶段将年龄四舍五入并裁剪到 0–100 范围，使用交叉熵损失进行监督；预测阶段对 logits 做 softmax 得到年龄概率分布，再计算概率分布的期望作为最终预测年龄。

设网络输出为 $f(x)$ ，年龄类别为 $a \in \{0, 1, \dots, 100\}$ ，则预测过程为：

$$P(a|x) = \text{softmax}(f(x)), \quad \hat{y} = \sum_{a=0}^{100} a \cdot P(a|x).$$

这种方法避免了只取最大概率类别时的离散跳变，也比直接回归单个年龄值更容易利用分类损失稳定训练。由于 APPA-REAL 的年龄标签来自多名标注者的表观年龄估计，标签本身具有一定主观性，使用概率分布表达预测结果更符合任务特点。

2.3 评价指标

实验主要使用平均绝对误差 MAE 评价模型：

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

MAE 的单位是岁，含义是模型平均预测偏差，因此比分类准确率更适合年龄估计任务。训练过程中每轮在验证集上计算 MAE，并以验证集 MAE 最小时的参数作为最终模型。

2.4 数据集与预处理

2.4.1 数据集

本实验使用 APPA-REAL 数据集（人脸裁剪后的 ‘_face.jpg’ 图像与相应 CSV 标注）。关键点：

- 输入尺寸：128 × 128 RGB；在数据加载器中通过 `cv2.resize(..., (128,128))` 强制为 128 分辨率；
- 标签：每张图像的 “apparent age”（取整并裁剪到 0–100）；训练时通过对真实标签加上带标准差的噪声进行数据增强（仅训练集启用）；
- 归一化：在 `FaceDataset` 中使用 `transforms.Normalize(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])`；
- 批量大小与并行：默认 `BATCH_SIZE=12`, `NUM_WORKER=8`。

项目要求将数据集整理为如下目录结构：

Listing 1: APPA-REAL 数据目录结构

```
DATASET/
|-- appa-real-release/
|   |-- gt_avg_train.csv
|   |-- gt_avg_valid.csv
|   |-- gt_avg_test.csv
|   |-- train/
|   |-- valid/
|   |-- test/
```

其中训练集和验证集 CSV 文件提供可用于监督学习的年龄标签；测试集没有公开真实年龄标签，因此训练结束后只能生成测试集预测文件，不能直接计算测试 MAE。

项目中的 `helperT.py` 负责读取数据。程序从训练、验证和测试三个 CSV 文件中读取图片文件名，再到对应的 `train`、`valid` 和 `test` 目录中查找 `_face.jpg` 人脸裁剪图。

项目还使用 `ignore_list.csv` 跳过部分异常样本，避免缺失图片或异常标注影响训练流程。

2.4.2 基础预处理

输入图片统一缩放为 128×128 ，并完成如下处理：

1. 使用 OpenCV 读取人脸图片；
2. 将图片 resize 到 128×128 ；
3. 将 BGR 格式转换为 RGB 格式；
4. 使用 ToTensor 转换为 PyTorch 张量；
5. 使用均值 $[0.5, 0.5, 0.5]$ 和标准差 $[0.5, 0.5, 0.5]$ 归一化，使输入大致落在 $[-1, 1]$ 。

2.4.3 训练集增强

为了提高模型对光照、姿态和成像质量变化的鲁棒性，训练集使用了多种数据增强：

1. 加性高斯噪声；
2. 高斯模糊；
3. 随机旋转，范围约为 -20° 到 20° ；
4. 小幅缩放和平移；
5. Gamma 对比度变化；
6. 随机水平翻转。

训练时还会根据 `apparent_age_std` 对年龄标签加入随机扰动，然后四舍五入并裁剪到 0–100 范围。这一处理利用了 APPA-REAL 标注标准差所反映的不确定性，有助于降低模型对单一整数标签的过拟合。

2.5 实现步骤

2.5.1 实现 AgeNet

AgeNet 是一个 VGG 风格的卷积神经网络，包含 5 个卷积特征提取阶段和 3 个全连接层。网络输入为 $3 \times 128 \times 128$ ，每经过一次 2×2 最大池化，空间尺寸减半，最终得到 $512 \times 4 \times 4$ 的特征图。

$128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$

展平后的 8192 维特征依次经过两个 4096 维全连接层和一个 101 维输出层，输出 0 到 100 岁的类别 logits。

表 1: AgeNet 网络结构

模块	层结构	输出尺寸变化
Block 1	Conv 7x7, 3->64 + ReLU + MaxPool	$128 \times 128 \rightarrow 64 \times 64$
Block 2	Conv 5x5, 64->128 + ReLU + Conv 5x5, 128->128 + ReLU + MaxPool	$64 \times 64 \rightarrow 32 \times 32$
Block 3	3 个 3x3 卷积, 通道 128->256->256->256 + MaxPool	$32 \times 32 \rightarrow 16 \times 16$
Block 4	3 个 3x3 卷积, 通道 256->512->512->512 + MaxPool	$16 \times 16 \rightarrow 8 \times 8$
Block 5	3 个 3x3 卷积, 通道 512->512->512->512 + MaxPool	$8 \times 8 \rightarrow 4 \times 4$
分类器	Flatten + 8192->4096->4096->101	输出 101 类

2.5.2 数据集加载、训练和验证

1. 编写数据加载器 `FaceDataset` 与增强管线 (使用 `imgaug`);
2. 在 `train_ageNet.py` 中构建训练循环 (100 epochs)、使用 SGD 优化器、交叉熵损失, 并在每个 epoch 后计算验证 MAE (以概率分布的期望作为年龄预测); 训练参数如表 2.

表 2: 训练参数设置

参数	设置
训练轮数	100 epochs
Batch size	12
优化器	SGD
学习率	0.001
Momentum	0.9
Weight decay	0.0
损失函数	CrossEntropyLoss
输入尺寸	128×128
输出类别	101 类, 对应 0 到 100 岁
最优模型选择	验证集 MAE 最小时保存

3. 在验证 MAE 下降时保存最佳模型权重到 `best_age_net.pth`; 训练完成后使用最佳模型生成测试集预测写入 `custom.txt`。

3 实验内容

本实验的主要任务为：基于给定网络结构与数据预处理，训练 AgeNet 实现年龄回归（通过 101 类分类后取期望作为连续年龄估计）。

3.1 模型结构与关键实现

关键模型定义摘自 `age_net.py`：

```
class AgeNet(nn.Module):
    def __init__(self, num_classes=101):
        super(AgeNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=7, padding=3), nn.ReLU(), nn.
                MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=5, padding=2), nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=5, padding=2), nn.ReLU(), nn.
                MaxPool2d(2),
            # 后续为多层 3x3 卷积与池化，最终通道 512
        )
        self.avgpool = nn.AdaptiveAvgPool2d((4, 4))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(512*4*4, 4096), nn.ReLU(), nn.
                Dropout(0.5),
            nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(4096, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = self.classifier(x)
        return x
```

实现要点说明：

- 使用 5 个卷积块逐步提取特征，保证训练时通道数与空间尺度与指导文档一致；
- 通过自适应池化确保进入全连接层前空间维为 4×4 ，从而使线性层维度固定；
- 输出为 101 维 logits，训练时使用 `CrossEntropyLoss`，评价时通过 softmax 得到概率分布，再计算期望作为年龄预测： $\hat{age} = \sum_{i=0}^{100} p_i \cdot i$ 。

3.2 训练流程（关键片段）

训练脚本中的核心循环（节选自 `train_ageNet.py`）：

```
for epoch in range(EPOCH):
    model.train()
    running_loss = 0.0
    for i, (y,x) in enumerate(train_loader):
        x = x.to(device).float(); y = y.to(device).long()
        outputs = model(x)
        loss = criterion(outputs, y)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        running_loss += loss.item()

    # validation: 用概率分布求期望并计算 MAE
    with torch.no_grad():
        for i, (y,x) in enumerate(val_loader):
            outputs = model(x.to(device).float())
            probs = torch.softmax(outputs, dim=1)
            pred_age = (probs * ages).sum(dim=1)
            predsVal.append(pred_age.cpu().numpy())
            gtsVal.append(y.cpu().numpy())
        mae = np.mean(np.abs(predsVal - gtsVal))
    print(f'Epoch {epoch+1}/{EPOCH}   loss={running_loss:.4f}   val_mae={
        mae:.4f}')
    if mae < best_mae:
        best_mae = mae
        torch.save(model.state_dict(), best_path)
```

关键超参数：EPOCH=100, TRAIN_LR=0.001, 优化器为 SGD（动量 0.9），无权重衰减。

4 实验结果

4.1 训练日志摘录

以下为训练过程中的关键输出片段（逐步记录验证 MAE 的变化与保存最佳模型信息）：

```
Using device: cuda
Epoch 1/100   loss=1667.0581   val_mae=19.0350
Saved best model to /home/rin/Local/Github/XJTU-Labs/Computer-Vision/z-
    final-project/best_age_net.pth
...
Epoch 51/100   loss=1270.1084   val_mae=9.9518
```



```
Saved best model to /home/rin/Local/Github/XJTU-Labs/Computer-Vision/z-
    final-project/best_age_net.pth
...
Epoch 74/100  loss=1214.7470  val_mae=7.9926
Saved best model to /home/rin/Local/Github/XJTU-Labs/Computer-Vision/z-
    final-project/best_age_net.pth
...
Epoch 89/100  loss=1194.1046  val_mae=7.3164
Saved best model to /home/rin/Local/Github/XJTU-Labs/Computer-Vision/z-
    final-project/best_age_net.pth
...
Epoch 100/100  loss=1172.5933  val_mae=7.4276
=> training finished
Test results saved to custom.txt
```

说明：

- "Using device: cuda" 表明训练时使用了 GPU，能显著加速卷积网络的训练；
- 训练早期（例如第 1 ~ 10 个 epoch）验证 MAE 快速从约 19 降到 11 ~ 12，表明网络从随机初始化开始快速学习到一定的年龄判别能力；
- 随后 MAE 持续缓慢下降，中后期（epoch 50 ~ 90）出现多个保存最佳模型的时刻，说明模型在验证集上不断找到更优的权重；
- 最佳记录出现在 epoch 89（val_mae = 7.3164），训练结束时最终验证 MAE 为 7.4276，整体表现稳定在约 7.3 ~ 7.9 范围；
- 每次出现"Saved best model to ..." 对应验证 MAE 刷新，这里保存路径为项目根下的 best_age_net.pth。

4.2 结果分析

- 从数值上看，最终验证 MAE 约为 7.3 ~ 7.4，这是从零初始化、基于 VGG 风格小型网络在 APPA-REAL 上能取得的合理基线精度；
- 模型以分类 + 期望回归的方式输出连续年龄估计：相比直接回归，分类后期望的做法可以通过 softmax 建模不确定性并稳定训练；
- 训练日志中 loss 为每个 epoch 的累加和（注意不是平均损失），因此与常见的每 batch 平均 loss 值在数量级上不同；
- 可进一步改进点：使用更强的数据增强、学习率调度（如 StepLR 或 CosineAnnealing）、预训练骨干（如 ImageNet 预训练的 VGG/ResNet）以及更大的批量或长时间训练。

5 总结

本实验按照指导实现并训练了 AgeNet 模型，验证集上最终 MAE 约为 7.3，训练过程中多次记录并保存了验证集上的最优权重，测试预测结果写入 `custom.txt`。实现遵循了实验要求：固定输入尺寸、5 个卷积块、3 个全连接层、SGD 优化及 100 个 epoch 上限。

6 附录：主要源码与运行命令

6.1 运行命令

在准备好 DATASET/ 后运行：

```
python train_ageNet.py
```

6.2 主要源码

6.2.1 age_net.py

```
import torch
import torch.nn as nn

class AgeNet(nn.Module):
    """AgeNet classification model producing 101-way logits (ages
        0..100).

    Designed for 128x128 RGB input following the provided architecture.
    """
    def __init__(self, num_classes=101):
        super(AgeNet, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=7, stride=1, padding=3),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(128, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(256, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(256, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(256, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
)

# ensure spatial size is 4x4 before the first FC
self.avgpool = nn.AdaptiveAvgPool2d((4, 4))

self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(512 * 4 * 4, 4096),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(4096, num_classes),
)

self._initialize_weights()
```

```
def forward(self, x):
    x = self.features(x)
    x = self.avgpool(x)
    x = self.classifier(x)
    return x

def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out',
                                    nonlinearity='relu')
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, 0, 0.01)
            nn.init.constant_(m.bias, 0)
```

6.2.2 train_ageNet.py

```
import os
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

from helperT import get_img_dataloaders, test_cel
from age_net import AgeNet

def train_ageNet(base_dir='DATASET/'):
    # hyper-parameters
    EPOCH = 100
    TRAIN_LR = 0.001
    MOMENTUM = 0.9
    WEIGHT_DECAY = 0.0

    # data
```

```
train_loader, val_loader, test_loader = get_img_dataloaders(
    base_dir)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

model = AgeNet(num_classes=101).to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=TRAIN_LR,
    momentum=MOMENTUM, weight_decay=WEIGHT_DECAY)
criterion = nn.CrossEntropyLoss()

best_mae = 1e9
best_path = os.path.join(os.path.dirname(__file__), 'best_age_net.
   .pth')

ages = torch.arange(0, 101).float().to(device)

for epoch in range(EPOCH):
    model.train()
    running_loss = 0.0
    for i, (y, x) in enumerate(train_loader):
        x = x.to(device).float()
        y = y.to(device).long()

        outputs = model(x)
        loss = criterion(outputs, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    # validation
    model.eval()
    predsVal = []
    gtsVal = []
    with torch.no_grad():
        for i, (y, x) in enumerate(val_loader):
            x = x.to(device).float()
```

```

        y = y.to(device).long()
        outputs = model(x)
        probs = torch.softmax(outputs, dim=1)
        pred_age = (probs * ages).sum(dim=1)
        predsVal.append(pred_age.cpu().numpy())
        gtsVal.append(y.cpu().numpy())

    predsVal = np.concatenate(predsVal, axis=0)
    gtsVal = np.concatenate(gtsVal, axis=0)
    mae = np.mean(np.abs(predsVal - gtsVal))

    print(f'Epoch {epoch+1}/{EPOCH}  loss={running_loss:.4f}
          val_mae={mae:.4f}')

    if mae < best_mae:
        best_mae = mae
        torch.save(model.state_dict(), best_path)
        print('Saved best model to', best_path)

    print('=> training finished')
    return best_path, test_loader

if __name__ == '__main__':
    best_path, test_loader = train_ageNet('DATASET/')

    # Load best model and run test to produce predictions file
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

    model = AgeNet(num_classes=101).to(device)
    model.load_state_dict(torch.load(best_path, map_location=device))
    model.to(device)

    # helperT.test_cel expects the model to be on cuda and will call .
    # cuda() on inputs.
    # If running on CPU, test_cel will still produce outputs if model
    # and inputs are on CPU.
    prediction = test_cel(model, test_loader, os.path.join(os.path.
        dirname(__file__), 'custom.txt'))
    print('Test results saved to custom.txt')

```